



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Лазар Нагулов

DSL и LSP за генерисање тест података

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Лазар Нагулов	Број индекса:	SV61/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

DSL и LSP за генерисање тест података

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultram dignissim convallis, quisque lobortis congue. Fusce id velit ut volutpat molestie, consectetur nulla eu dolor porta. Vestibulum ante ipsum primis in faucibus orci luctus et ultram posuere. Cras mattis consectetur purus sit amet fermentum. Aenean lacinia bibendum nulla sed consectetur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Лазар Нагулов
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	DSL и LSP за генерисање тест података
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/33/12/0/12/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Lazar Nagulov
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	DSL and LSP for test data generation
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/33/12/0/12/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	1
1.2	Предмет и цињ рада	1
1.3	Допринос рада	1
1.4	Структура рада	1
2	Теоријске основе	3
2.1	Језици специфични за домен	3
2.2	Фазе обраде програмског језика	3
2.3	Међурепрезентације	3
2.4	Систем модула	3
2.5	Протокол језичких сервера	3
3	Анализа постојећих решења	5
3.1	Језици специфични за домен	5
3.2	Системи за генерисање тест података	5
3.3	Приступи имплементацији језичке подршке	5
4	Дизајн решења	7
4.1	Архитектура система	7
4.2	Дизајн језика специфичног за домен	8
4.3	Дизајн семантичке анализе	10
4.4	Дизајн високонивојске међурепрезентације	10
4.5	Дизајн модула	11
4.6	Дизајн језичког сервера	12
5	Имплементација	15
5.1	Лексичка анализа	15
5.2	Парсирање	15
5.3	Апстрактно синтаксно стабло	15
5.4	Семантичка анализа	15
5.5	Високонивојска међурепрезентација	15
5.6	Систем модула	15
5.7	Језички сервер	15
6	Резултати и дискусија	17
6.1	Тестирање језика	17
6.2	Тестирање резултата модула	17
6.3	Тестирање језичког сервера	17
6.4	Дискусија резултата	17
7	Закључак	19
8	Будући рад	21
8.1	Средњенивојска међурепрезентација	21
	Списак слика	23
	Списак листинга	25
	Списак коришћених скраћеница	27
	Списак коришћених појмова	29
	Биографија	31
	Литература	33

1.1 Мотивација

1.2 Предмет и цињ рада

1.3 Допринос рада

1.4 Структура рада

2.1 Језици специфични за домен

2.2 Фазе обраде програмског језика

2.3 Међурепрезентације

2.3.1 Апстрактно синтаксно стабло

2.3.2 Високонивојска међурепрезентација

2.4 Систем модула

2.5 Протокол језичких сервера

2.5.1 Проблем развоја језичке подршке

2.5.2 Архитектура протокола језичког сервера

2.5.3 Функционалности протокола језичког сервера

Глава 3

Анализа постојећих решења

3.1 Језици специфични за домен

3.2 Системи за генерисање тест података

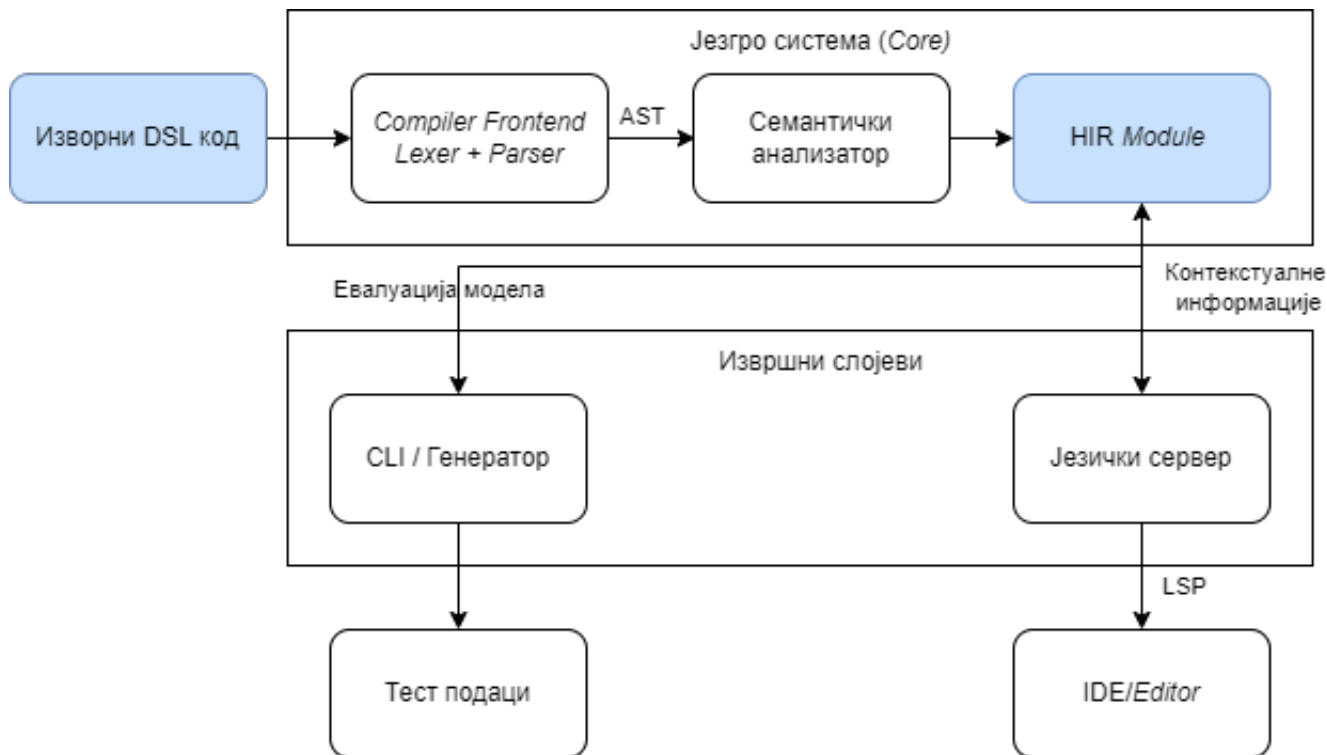
3.3 Приступы имплементацији језичке подршке

Циљ софтверског решења је развој DSL-а који омогућава генерисање тест података. Систем као улаз прима текст DSL датотеке и захтеве које *едитор* шаље путем LSP протокола. Излаз система чине текстуалне датотеке са изгенерисаним тест подацима и одговори формирану у складу са LSP спецификацијом.

Ово поглавље описује целокупан процес дизајна система. Архитектура система (поглавље 4.1) приказује глобалну структуру софтвера и међусобне везе између главних компоненти. Наредни сегмент, посвећен језику специфичном за домен (поглавље 4.2), дефинише конкретну синтаксу, граматичка правила и припадајуће структуралне елементе. Семантичка анализа (поглавље 4.3) обухвата начин имплементације правила контекстуалне исправности и процес валидације изворног кода. За премошћавање јаза између текстуалног записа и саме генерације тест података задужена је високонивојска међурепрезентација (поглавље 4.4), која уводи специфичан апстрактни модел. Рашчлањивање унутрашње структуре софтвера на независне компоненте уз дефинисање њихових интерфејса изложено је у оквиру дизајна модула (поглавље 4.5). Напослетку, део о језичком серверу (поглавље 4.6) описује архитектуру подсистема за прихватање LSP захтева, управљање стањем отворених докумената и формирање стандардизованих одговора за развојно окружење.

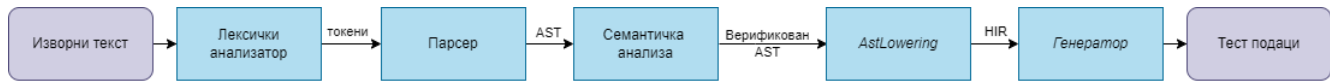
4.1 Архитектура система

Архитектура система пројектована је по принципу дељеног језгра. Логика лексичке, синтаксне и семантичке анализе користи у различитим модулима извршавања. Глобална структура софтвера и међусобне везе између главних компоненти приказане су на слици 1.



Слика 1: Дијаграм архитектуре система

Процес преводјења изворне спецификације у финални извештај одвија се кроз неколико узастопних фаза. Ток података и постепена трансформација модела кроз ланац обраде (енг. *pipeline*) у компајлеру илустровани су на слици 2.



Слика 2: Ланац обраде у компајлеру

4.2 Дизајн језика специфичног за домен

Дизајн DSL-а се заснива на декларативном принципу. Корисник описује структуру и ограничења података. Поступак генерисања је потпуно апстрахован. Систем је пројектован као статички типизиран језик. Статичка типизација омогућава рано откривање грешака у спецификацији путем језичког сервера.

4.2.1 Директиве и конфигурација контекста

Синтаксне директиве почињу симболом `@`. Директиве служе за глобалну конфигурацију окружења и управљање модулима. Ови конструкти дефинишу метаподатке преводиоца. Пример глобалних директива и увоза модула приказан је на листингу 1.

```

@import "skalarne_definicije.tmod";

@output json { pretty = true; indent = 4; }
@output_path "./izlaz/korisnici.json";

@generate Korisnik[100];
  
```

Листинг 1: Пример глобалних директива и увоза модула

Директива `@import` омогућава декомпозицију комплексне спецификације на мање модуле. Директива `@output` дефинише излазни формат JSON (енг. *JavaScript Object Notation*) и пратећу индентацију. Директива `@output_path` одређује циљну локацију датотеке на диску. Извршна директива `@generate` покреће генерисање над изабраним шаблоном.

4.2.2 Кориснички типови и синтаксна ограничења

Систем типова подржава проширење примитивних типова кроз увођење корисничких типова са ограничењима. Дефинисање скаларних типова са ограничењима илустровано је на листингу 2.

```

type GodinaStudija = int [range=1..=4];
type ProsecaOcena = float [range=6.0..=10.0];
type BrojIndeksa = string_pattern "2026/${#[4]}";
  
```

Листинг 2: Дефинисање скаларних типова са ограничењима

Синтаксна конструкција `range` ограничава домен дозвољених нумеричких вредности. Генератор примењује кориснички дефинисану униформну дистрибуцију вредности унутар задатог интервала. Спецификација `string_pattern` користи текстуалне макрое за форматирање псеудо-насумичних низова карактера.

4.2.3 Колекцијски типови и вишедимензионалне структуре

Систем типова подржава дефинисање колекција кроз типске листе. Листе омогућавају груписање више елемената истог базног типа. Број генерисаних елемената контролише се синтаксним ограничењем `count`. `Count` дефинише минималан и максималан број чланова колекције. Дефинисање колекцијских типова и уложених структура приказано је на листингу 3.

```

type Grade = int [range=6..10];
type IntList = [int][count=1..5];
type IntMatrix = [IntList][count=1..5];

template Student {
    grades = [Grade][count=1..5];
    tensor = [IntMatrix][count=1..5];
}

```

Листинг 3: Дефинисање колекцијских типова и вишедимензионалних структура

Конструкт листе прихвата примитивне и кориснички дефинисане скаларне типове. Типски систем дозвољава вишеструко улагање листа. Улагање омогућава креирање сложених вишедимензионалних структура попут матрица и тензора. Генератор стохастички одређује коначну дужину сваке листе унутар дефинисаног интервала приликом инстанцирања шаблона.

4.2.4 Пробабилистичке енумерације

Енумерације дефинишу коначан скуп вредности и припадајуће тежинске факторе дистрибуције. Тежински фактори се експлицитно наводе помоћу оператора `=>`. Пример енумерације са факторима тежине дистрибуције приказан је на листингу 4.

```

enum StatusStudenta {
    Budzet          => 7;
    Samofinansiranje => 3;
}

```

Листинг 4: Енумерација са факторима тежине дистрибуције

Генератор користи дефинисане тежине за стохастичко узорковање вредности. Вероватноћа избора ставке буџет износи тачно 70% док за самофинансирање износи 30%. Пробабилистичке енумерације омогућавају подешавање статистичког профила излазних података.

4.2.5 Структурна композиција и наслеђивање

Моделовање сложених објеката извршено је кроз концепте структура (енг. struct) и шаблона (енг. template). Структура дефинише чист логички контејнер за композицију поља. Шаблон представља активну семантичку јединицу над којом се извршава генерација. Наслеђивање поља дефинише се оператором `:`. Структурна композиција и механизам наслеђивања приказани су на листингу 5.

```

struct Lokacija {
    grad    = string;
    drzava  = string;
}

template Osoba {
    ime     = string;
    adresa  = Lokacija;
}

template Zaposleni : Osoba {
    plata   = int [range=50000..150000];
}

```

Листинг 5: Структурна композиција и механизам наслеђивања

Шаблон `Zaposleni` аутоматски наслеђује сва поља родитељског шаблона `Osoba`. Наслеђивање обухвата и уложено поље `adresa`. Наслеђивање смањује редундантност приликом спецификације сложених домена. Композиција омогућава хијерархијску организацију података унутар резултујућег записа.

4.2.6 Очување релационог интегритета

Очување логичких веза између генерисаних ентитета реализовано је увођењем кључне речи *ref*. Референцирање вредности ради очувања интегритета илустровано је на листингу 6.

```
template Smer {
    sifra = string_pattern "${A[3]}";
}

template Student {
    id      = int;
    smer_kod = ref Smer.sifra;
}
```

Листинг 6: Референцирање вредности ради очувања интегритета

Кључна реч *ref* сигнализира преводиоцу да вредност поља мора бити преузета из скупа већ изгенерисаних вредности циљног шаблона. Очување релационог интегритета симулира спољне кључеве из релационих база података. Спољни кључеви спречавају појаву неконзистентних података у финалном извештају.

4.3 Дизајн семантичке анализе

Семантичка анализа представља средишњи део аналитичког слоја унутар архитектуре система. Синтаксни анализатор прослеђује изграђен AST семантичком слоју. Семантички анализатор обавља три фазе провере. Излаз ове компоненте обухвата попуњену табелу симбола и скуп дијагностичких порука.

Изградња табеле симбола чини почетну фазу семантичке анализе. Компонента за изградњу табеле симбола пролази кроз структуру стабла помоћу дизајнерског шаблона посетилац (енг. *visitor*). Модул региструје све дефинисане шаблоне, структуре, енумерације и корисничке типове. Компонента прати контекст извршавања кроз улазак у опсеге видљивости и излазак из њих. Систем детектује невалидне контексте попут дефинисања поља изван дозвољених структура. Анализатор проналази дуплиране идентификаторе унутар истог опсега.

Провера типова представља другу фазу семантичке анализе. Модул за проверу типова анализира исправност израза и додељених вредности. Компонента утврђује конкретан тип за сваки конструкт у спецификацији. Систем захтева да тежински фактори унутар енумерација буду искључиво целобројне вредности. Компонента такође контролише број генерисаних ентитета у извршним директивама. Наведени број мора одговарати целобројном типу података. Свако одступање од ових правила резултује генерисањем семантичке грешке о неслагању типова.

Провера референци је завршна фаза семантичке анализе. Компонента контролише постојање свих коришћених идентификатора. Компонента потврђује исправност кључне речи *ref* приликом повезивања поља између различитих шаблона, претражује локалну табелу симбола и табеле симбола из увезених модула. Провервач референци детектује непостојеће родитељске шаблоне у процесу наслеђивања. Фаза очувања интегритета спречава позивање непостојећих поља унутар структура.

4.4 Дизајн високонивојске међурепрезентације

Процес преводјења из AST-а у HIR изводи компонента *AstLowering* [1]. *AstLowering* компонента уклања синтаксно-декоративне елементе и трансформише структуре у облике погодне за генерацију података и рад језичког сервера.

HIR обухвата интернирање стрингова, адресирање ентитета, спуштање израза и издвајање ограничења. Басен стрингова (енг. *string pool*) преводи текстуалне идентификаторе у нумеричке кључеве типа *StringId*. Поређење стрингова се тиме своди на операције над целим бројевима. Униформно адресирање обезбеђује јединствене кључеве на нивоу модула. Ентитети користе *ItemId* за глобалне ставке и *FieldId* за поља шаблона или структура. Увођењем нумеричких идентификатора, целокупан семантички граф програма

постаје независан од оригиналног текстуалног записа. Независност омогућава ефикасну претрагу графа и брзу бинарну серијализацију.

Крајњи резултат процеса спуштања је структура *Module* (листинг 7). *Module* служи као централни контејнер који обједињује све релевантне аспекте једне компајлиране датотеке.

```
struct Module {
    metadata: ModuleMetadata,
    string_pool: StringPool,
    imports: Vec<StringId>,
    directives: Vec<Directive>,
    items: Vec<Item>,
    source_map: SourceMap,
}
```

Листинг 7: Централна структура HIR модула

Module структура концептуално повезује метаподатке, басен стрингова, увозне зависности, извршне директиве, компајлиране ставке и мапу изворног кода. Модул се посматра као јединствена, самодовољна целина, погодна за пренос између различитих фаза компајлера.

4.5 Дизајн модула

Дизајн модуларног система инспирисан је архитектуром изолованих метаподатака компајлера *rustc* [2]. Архитектура омогућава организацију дефиниција кроз више датотека и симултани рад језичког сервера са више табела симбола. Систем почива на три дизајнерске компоненте:

- међумодуларно референцирање (поглавље 4.5.1),
- перзистентност и серијализација (поглавље 4.5.2) и
- пеконструкција стања за језички сервер (поглавље 4.5.3).

4.5.1 Међумодуларно референцирање

Повезивање симбола између различитих модула захтева модел који разликује локалне ставке од оних које су увезене из спољних датотека. За ову намену дизајнирана је енумерација *ItemRef* (листинг 8).

```
enum ItemRef {
    Local(ItemId),
    Imported { module: StringId, item: ItemId },
}
```

Листинг 8: Енумерација за представљање локалних и увезених референци

Варијанта *Local* користи се када се референцирани ентитет налази унутар истог модула. Варијанта *Imported* омогућава унакрсно референцирање тако што експлицитно чува идентификатор спољног модула и идентификатор саме ставке. Процес разрешавања назива претражује локалне идентификаторе пре него што претрагу преусмери на мапу увезених зависности.

4.5.2 Перзистентност стања

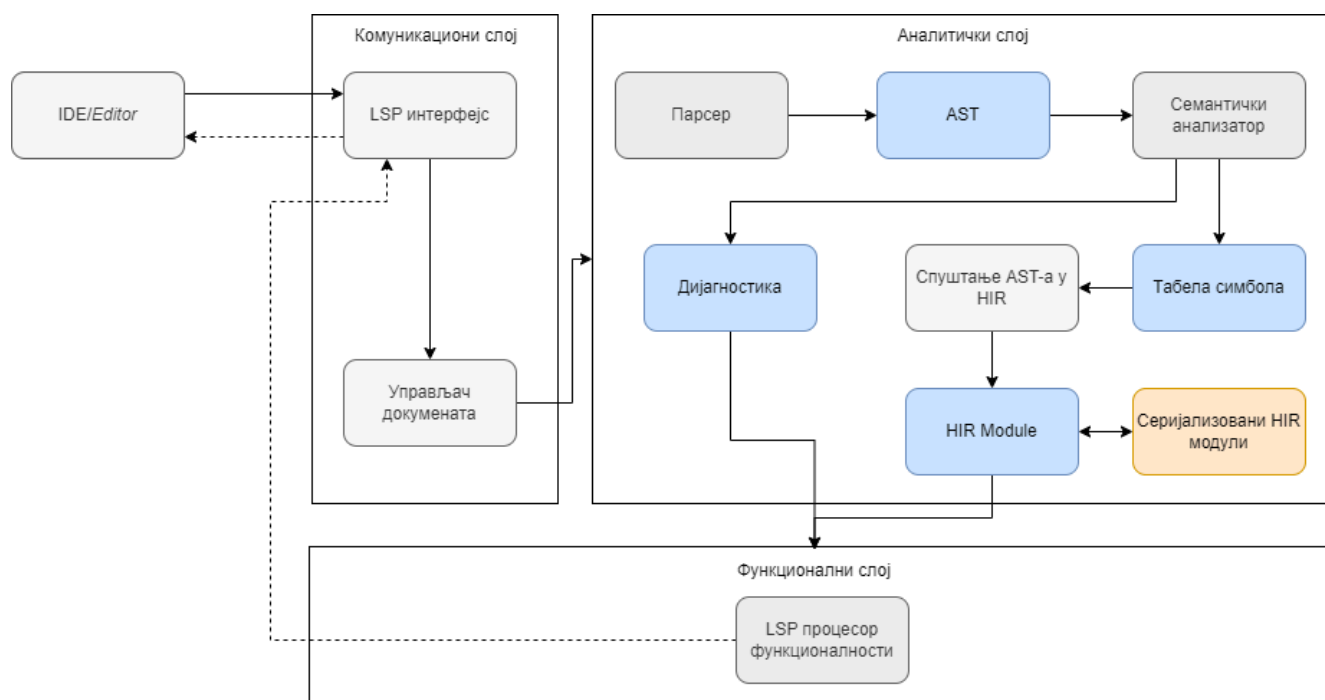
Да би се избегло поновно парсирање и семантичка анализа зависних датотека, дизајн предвиђа чување анализираног HIR модела на диску. Као формат серијализације изабран је *MessagePack* [3]. *MessagePack* формат обезбеђује компактан бинарни запис и брзу десеријализацију [3]. Централна структура *Module* пројектована је тако да буде потпуно самодовољна приликом уписа и читавања са диска.

4.5.3 Реконструкција стања за језички сервер

За потребе рада језичког сервера, дизајн обухвата механизам трансформације статичког HIR модела назад у динамичку табелу симбола. Процес захтева итерацију кроз све компајлиране ставке унутар модула и поновно мапирање њихових својстава. Реконструкција омогућава језичком серверу да креира контекст изолованих датотека и изврши ефикасну валидацију међумодуларних веза.

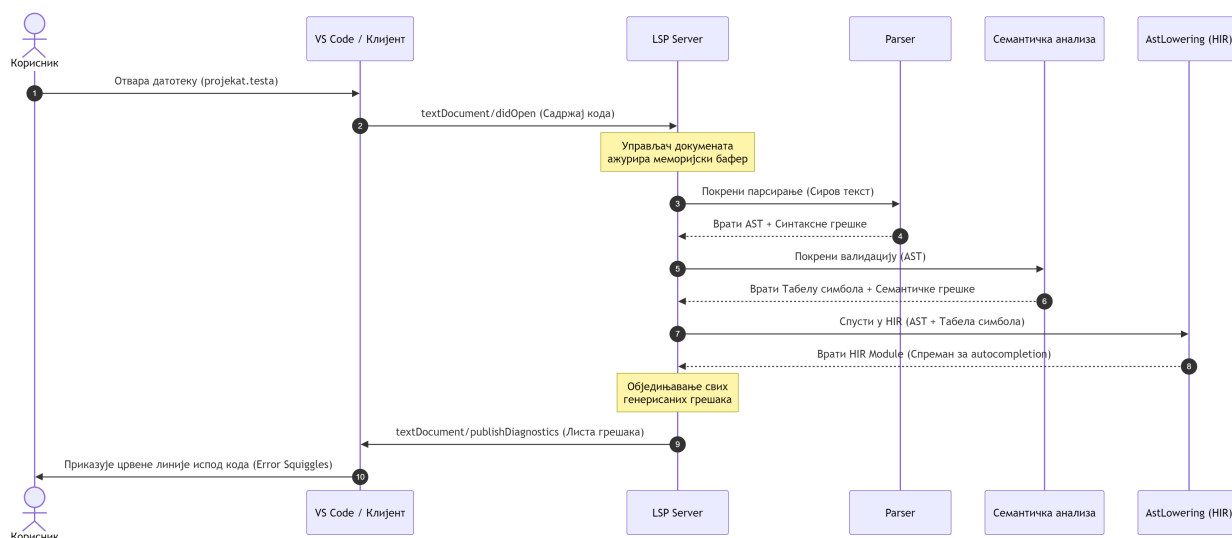
4.6 Дизајн језичког сервера

Језички сервер обухвата три логичка слоја: комуникациони слој (поглавље 4.6.1), аналитички слој (поглавље 4.6.2) и функционални слој (поглавље 4.6.3). Слика 3 приказује компоненте сваког слоја и ток обраде LSP захтева.



Слика 3: Архитектура LSP система и ток обраде LSP захтева.

LSP се заснива на асинхроној размени порука које су вођене спољним догађајима унутар развојног окружења. Временски редослед размене асинхроних порука и след активности унутар система приликом отварања датотеке приказани су на слици 4.



Слика 4: Секвенцијални дијаграм обраде догађаја при отварању датотеке

4.6.1 Комуникациони слој

Комуникациони слој представља улазну тачку система. Задужен је за размену порука између *едитора* и језичког сервера. Слој прима LSP захтеве, управља стањем отворених докумената и прослеђује поруке осталим компонентама система [4].

Улазну тачку комуникационог слоја представља LSP интерфејс. Компонента прихвата захтеве које *едитор* шаље и преводи их у интерне операције система [4]. Након обраде, компонента формира одговор и враћа га *едитору* [4].

Поред LSP интерфејса, комуникациони слој садржи управљач докумената (енг. *document manager*). Компонента прати измене над отвореним документима и одржава њихово стање за потребе даље обраде. Свака измена садржаја документа покреће поновну анализу и ажурирање дијагностичких порука.

4.6.2 Аналитички слој

Аналитички слој врши обраду DSL докумената и припрема податке потребне за LSP функционалности. Слој обухвата парсирање (енг. *parsing*), семантичку анализу и изградњу интерних модела података. Резултат анализе представља HIR, која садржи семантички модел програма независан од синтаксног записа. Кроз серијализацију табеле симбола, HIR омогућава подршку за независне модуле. Језичком серверу HIR пружа могућност симултаног приступа већем броју међусобно повезаних табела симбола. HIR се користи као заједничка основа за LSP функционалности, генерацију тест података и серијализацију модула.

Обрада започиње парсером. Парсер трансформише текст документа у AST [1]. У случају уочавања синтаксних неправилности, парсер примењује механизме опоравка од грешака како би наставио са обрадом остатка документа. Овакав приступ омогућава систему да идентификује и прикаже више независних грешака истовремено, уместо прекида обраде при првој грешци [1].

Над генерисаним AST-ом извршава се семантичка анализа. Компонента проверава типове, валидност референци и конзистентност конструкција дефинисаних DSL правилима. Током анализе, формира се локална табела симбола, која садржи дефиниције и везе између елемената програма [1]. Уочене неправилности се бележе кроз модел дијагностике. Модел садржи опис грешке, ниво озбиљности и локацију у документу [4].

Након успешно завршене семантичке анализе AST се трансформише у HIR. У HIR-у су разрешене референце, типови и односи између симбола, како из тренутног документа, тако и из претходно серијализованих увозних модула. Ово HIR чини погодним за трајно чување и поновно учитавање модуларних целина.

4.6.3 Функционални слој

Функционални слој имплементира кориснички видљиве операције трансформацијом интерних модела у одговоре дефинисане LSP спецификацијом. Слој обухвата компоненте које омогућавају:

- аутоматско допуњавање кода на основу тренутног контекста и симбола доступних у локалној табели симбола и унутар серијализованих HIR модела увезених модула,
- дијагностику грешака, коришћењем резултата семантичке анализе за формирање порука о неправилностима,
- навигацију кроз код која кориснику обезбеђује прелазак на дефиниције симбола кроз глобални опсег дефинисан учитаним табелама симбола и
- приказ додатних информација (енг. *hover*), пружањем контекстуалних описа интеракције са кодом.

5.1 Лексичка анализа

5.2 Парсирање

5.3 Апстрактно синтаксно стабло

5.4 Семантичка анализа

5.5 Високонивојска међурепрезентација

5.6 Систем модула

5.7 Језички сервер

6.1 Тестирање језика

6.2 Тестирање резултата модула

6.3 Тестирање језичког сервера

6.4 Дискусија резултата

8.1 Средњенивојска међурепрезентација

Списак слика

Слика 1	Дијаграм архитектуре система	7
Слика 2	Ланац обраде у компајлеру	8
Слика 3	Архитектура LSP система и ток обраде LSP захтева.	12
Слика 4	Секвенцијални дијаграм обраде догађаја при отварању датотеке	12

Списак листинга

Листинг 1	Пример глобалних директива и увоза модула	8
Листинг 2	Дефинисање скаларних типова са ограничењима	8
Листинг 3	Дефинисање колекцијских типова и вишедимензионалних структура	9
Листинг 4	Енумерација са факторима тежине дистрибуције	9
Листинг 5	Структурна композиција i механизам наслеђивања	9
Листинг 6	Референцирање вредности ради очувања интегритета	10
Листинг 7	Централна структура HIR модула	11
Листинг 8	Енумерација за представљање локалних и увезених референци	11

Списак коришћених скраћеница

Скраћеница	Опис
AST	Abstract Syntax Tree (Апстрактно синтаксно стабло)
DSL	Domain Specific Language (језик специфичан за домен)
GPL	General Purpose Language (језик опште намене)
JSON	JavaScript Object Notation
IDE	Integrated Development Environment (интегрисано развојно окружење)
LSP	Language Server Protocol (протокол језичких сервера)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007.
- [2] The Rust Project Developers, “Guide to rustc Development.” Accessed: July 7, 2026. [Online]. Доступно на <https://rustc-dev-guide.rust-lang.org/>
- [3] S. Furuhashi, “MessagePack Specification.” Accessed: July 7, 2026. [Online]. Доступно на <https://msgpack.org/>
- [4] Microsoft, “Language Server Protocol Specification (Version 3.17).” Accessed: April 29, 2026. [Online]. Доступно на <https://microsoft.github.io/language-server-protocol>